

Abstract

At the core of our toolpath generation framework lies a forward simulation model that replicates material removal, i.e. the carving of a workpiece by a rotating tool along a G-code trajectory. This simulator must be fast, as it is designed to be queried repeatedly within optimization loops etc., and accurate, since its outputs provide the training signal for a learning-based component.

We introduce the GPU carving pipeline of *autocam*, a 3-axis milling simulator on OpenGL compute shaders, using a single-axis dixel field: a grid of columns storing sorted Z-transitions, where Boolean subtraction reduces to an embarrassingly parallel per-column merge, with minimal synchronization overhead.

To isolate our contributions, we benchmark against two established baselines that represent the state of the art in GPU dixel simulation: (i) per-step stamping [1] and (ii) swept-volume subtraction via a separate external buffer [2, 3, 4]. Against these, we propose two complementary contributions:

Fused swept-segment subtraction: for each linear toolpath segment, the swept-tool column is computed entirely on the GPU, on the fly as a Z-envelope and subtracted in place, never materializing the swept volume. Because each column is independent, the operation is embarrassingly parallel and maps directly to one GPU thread per column. Compared to the external-buffer swept baseline (which materializes the swept volume in a separate buffer and subtracts it in a second pass), fusion is $\approx 1.9\times$ faster and halves resident memory, a consistent win across all workloads.

Tube pruning: Because the GPU dispatch covers the axis-aligned bounding box of a segment's endpoints-far larger than the actual swept band on diagonal moves and in-air rapids, we skip write-back for columns the tool never touches. This byte-identical optimization delivers 2.9–7.6 $\times$ on the stress benchmarks (Table 1), at a cost of at most $\sim 10\%$ on axis-aligned programs, where the dispatch already matches the swept band.

Two documented negative results -a range-query envelope acceleration and an oriented tiled dispatch- corroborate our central diagnosis: on this pipeline, the lever that pays is the one that removes memory-write traffic, not the ones that reduce compute or launched threads. The benchmarks span axis-aligned contours, 45° rasters, and air-heavy programs; the three swept levels are byte-

identical to one another and approximate the continuously-swept cut to within a measured under-removal below 0.2 %.

Keywords: NC machining simulation, material removal, dixel, swept volume, GPU compute, memory bandwidth, 3-axis milling.

1. Introduction

1.1 Motivation

Public CNC toolpath data is scarce because it is proprietary. Our long-term goal is to synthesize training data for a learned toolpath generator: given a target part, a stock block, and a cutting tool, produce the G-code that machines the part. Learning this mapping requires many G-code-to-machined-geometry pairs, which in turn demand a fast and accurate forward model: a simulator that, given a toolpath, tool, and stock, computes the resulting workpiece.

This forward model sits inside an optimization loop (e.g., a genetic algorithm that searches over toolpaths and scores each candidate against the target); it may therefore be evaluated millions of times per optimization run. Raw simulation speed at fixed accuracy is thus the central engineering requirement, not merely one of many, and the primary driver of our design choices; the model’s geometric error against the continuously-swept cut is small — a measured under-removal below 0.2 %, quantified in §4.6 — so accuracy here is a constraint to hold, not the quantity we optimize. We restrict our scope to 3-axis milling (tool translation only, with the tool axis fixed to Z).

1.2 Related work

Column-based material-removal simulation begins with the dixel model [1], which renders NC milling in real time as an image-space Boolean subtraction: the workpiece is a grid of Z-direction “depth elements” (dexels), and the tool lowers their heights. The model was extended to multi-/tri-dixel representations (three orthogonal dixel bundles) to improve sampling on steep walls: [2] present online sculpting on multi-dixel volumes, and GPU variants were developed [3, 4]. In parallel, the solid-modeling community proposed sparse, per-axis implicit representations with GPU Booleans, notably Layered Depth-Normal Images, LDNI [5]. Swept-volume computation has its own line, from the sweep-envelope differential equation [6] to the survey [7]. Surface reconstruction uses Marching Cubes [8], and its GPU version [9].

Two observations frame our contribution. First, subtracting a tool’s swept volume per move from a dixel model is not new - it is exactly what [2,3,4] do. Second, once the obvious per-step overheads are removed, such a simulator is limited not by geometry but by memory bandwidth on the working-set buffer. Our contribution is therefore not a new representation, but (i) an efficient *materialization-free, single-axis* GPU realization of swept-segment subtraction, and (ii) tube pruning, an optimization that targets memory-write traffic - the actual bottleneck - which the prior GPU-dixel line does not address.

1.3 Contributions

We present two main contributions, both implemented entirely on the GPU. Each is isolated by A/B comparison against the best available baseline from the literature, and each is byte-identical to the swept baseline it builds on (S0 is a timing and behavioural reference, not a geometric ground truth; §4.6).

Contribution 1: Fused swept-segment subtraction. For each linear toolpath segment, the swept-tool column is computed on the fly as a Z-envelope and subtracted in place, never materializing the swept volume. Because columns are independent, the operation maps directly to one GPU thread per column. Compared to an external-buffer swept baseline (which writes the swept volume to a separate buffer and subtracts it in a second pass), fusion is $\approx 1.2\text{--}2.5\times$ faster (geomean $\approx 1.9\times$) and halves resident memory: the external buffer requires a full-size twin of the working set that the fused kernel eliminates (§6.1, §6.3). This is a consistent win across all workloads.

Contribution 2: Tube pruning with in-air skip. The GPU dispatch covers the axis-aligned bounding box of each segment’s endpoints, which on diagonal moves and in-air rapids is far larger than the actual swept band. We skip the per-column write-back for columns the tool never touches, leaving them untouched and the result byte-identical. Tube pruning delivers $2.9\text{--}7.6\times$ on the stress benchmarks (Table 1), while remaining near-neutral on axis-aligned programs (a 7–11 % cost, where the dispatch already matches the swept band) (§6).

Beyond these two main contributions, we introduce two additional refinements that improve specific aspects of the pipeline. First, we perform GPU-side result compaction, which reduces read-back traffic by approximately an order of magnitude by returning only the compact transition lists rather than the full unpacked buffer (§3.5). Second, we implement a sparse tiled backend that materializes only the tiles actually touched during carving. This is a memory/scale lever rather than a speed optimization: for localized programs, it

shrinks the resident footprint to a small fraction of the full buffer; for whole-stock machining, however, it offers no saving. The backend is runtime-selectable and byte-identical to the flat buffer version (§6.2).

We also report two attempted optimizations that we implemented in full and then discarded because they did not pay off. The first was a range-query envelope acceleration: we precomputed per-tool directional Z-envelopes as sparse tables, turning the per-column envelope computation from an $O(\text{sub-steps})$ loop into an $O(1)$ query. This gave only modest improvement, because it optimized compute, which is not the bottleneck, and was reverted. The second was an oriented tiled dispatch: we attempted to tile the launch along the swept band itself, so that off-band threads would not even be launched. This required barriers between tiles to preserve byte-identity, and the resulting synchronization overhead, together with overlapping joins between adjacent tiles, made the approach slower than tube pruning alone. We reverted it as well. Together, these negative results corroborate our central diagnosis: on this pipeline, the lever that pays is the one that removes memory-write traffic, not the ones that reduce compute or launched threads (§7).

2. Volume representation: single-axis dixel field

Both the stock and the tool are discretised on a common voxel lattice of $W \times H \times D$ cells, but that lattice is never materialised: it only fixes the sampling resolution and the integer coordinates in which every quantity is expressed. What is actually stored is a single-axis dixel field along Z. Each (x, y) column of the lattice is a one-dimensional occupancy function of z , encoded as the sorted list of the heights at which occupancy flips: empty to solid or solid to empty. Because occupancy alternates, transitions come in pairs, one entering and one leaving the solid.

Milling parts are massive bodies bounded by a few surfaces, so a column crosses the boundary only a handful of times: two to four transitions is typical, and two is the minimum, since transitions come in pairs. An uncarved column of a prismatic stock is exactly that minimum, the single pair $[0, z_{\text{top}}]$, where z_{top} is the height of the stock's top face: solid from the base up to that face, empty above.

Z-transition column representation (single-axis dixel)

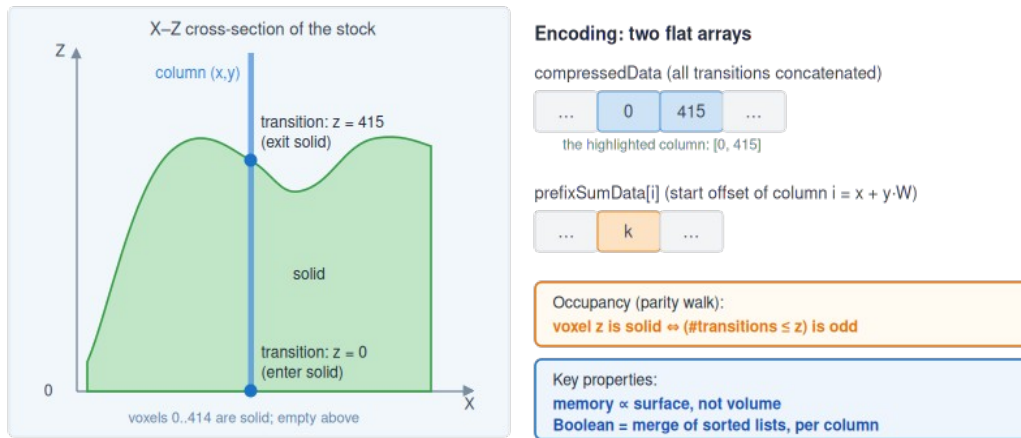


Figure 1. The single-axis dixel (Z-transition column) representation. A column stores the sorted heights at which occupancy flips; two flat arrays encode the whole object, and occupancy at any voxel is recovered by a parity walk. Storage scales with the complexity of the boundary rather than with the enclosed volume, and any Boolean operation becomes a merge of sorted lists, independent per column.

The whole object is a struct `VoxelObject`, used for both stock and tool, consisting of two flat arrays. Columns are addressed by the linear index $i = x + y \cdot W$, i.e. row-major with X varying fastest, and both arrays follow that order:

- `compressedData`: the transition lists of all columns, each sorted > ascending, concatenated in order of i .
- `prefixSumData`: for each column i , the offset at which its list > begins in `compressedData`. The array has one entry per column, so > the length of column i 's list is `prefixSumData[i+1] - > prefixSumData[i]`, with the last column running to the end of > `compressedData`. During a carving session the resident stock keeps > its per-column counts in a separate array instead (§4.2).

The name is historical: the struct stores transitions, not voxels.

Occupancy at (x, y, z) is recovered by a parity walk: the voxel is solid iff the number of transitions of its column that are $\leq z$ is odd. No dense buffer is ever allocated, read or written.

Three properties are exploited throughout.

1. **Compactness:** storage is proportional to the number of transitions, that is, to how often the columns cross the boundary, and not to the enclosed volume. A field averaging c transitions per column costs $c \cdot W \cdot H$ integers where a dense grid would cost $W \cdot H \cdot D$ occupancy values, a saving of a factor D/c . For $c \approx 2-4$ and the depths typical of a milling stock this is two to three orders of magnitude, and it does not grow as the part becomes more massive, only as its surface becomes more intricate (absolute figures for the reference stock used in the experiments are given in §4.2 and §6.3).
2. **Boolean = per-column merge.** Any Boolean between two fields on the same lattice is a two-pointer merge of their sorted transition lists, performed independently for each of the $W \cdot H$ columns: the traversal is the same in every case, only the predicate on the two occupancy states differs (AND for intersection, OR for union, AND-NOT for difference), and a transition is emitted wherever the predicate changes value. The carver uses difference alone (subtracting the tool from the stock) and it is the only operation implemented on the GPU (§4.4). Columns never interact, so the merge is embarrassingly parallel and maps to one GPU thread per column (§4.1).
3. **A free bounding interval.** The lowest and highest transitions of a column are exactly its solid extent [bottom, top], available without a search. For tools whose columns are a single Z-interval (§4.5) that interval is the column, which is what makes the on-the-fly Z-envelope of §3.3 possible.

3. From naive stamping to the final solution

We build the method as a sequence of refinement levels, each validated against the previous one (byte-identical among the swept levels; §4.6). To make the transformations concrete we follow one setting throughout: a prismatic stock, a ball-end mill i.e. a cylinder closed by a hemispherical tip of the same radius, and a single linear segment carrying the tool from p_0 to p_1 . The tool is positioned by the centre of its own bounding box: the G-code coordinate, converted to voxel units, places that centre, with no compensation for the tip radius or the shank length. This is a calibration convention, not a modelling choice — it applies identically to every refinement level, so it affects none of the comparisons below, but it is why a calibrated millimetre mapping is listed among the limitations (§7). Dimensions are left unspecified, since none of the steps depends on them; the measured configurations are those of §5.

3.1 Naive stamping (baseline)

The naive simulator advances the tool along the segment in fixed jog steps and stamps the whole tool at each of them — one Boolean-difference dispatch per step, subtracting the tool at that position from the stock. The step s is a fixed distance in voxel units along the segment (default $s = 2$, settable at run time), independent of the segment's direction and length. A segment of length L thus costs $\approx L/s$ dispatches. Two things are wasteful. The dispatches overlap: consecutive positions differ by s while the tool footprint is many voxels wide, so every column under the tool is read, merged and rewritten several times over, though only the first of those passes usually removes anything. And the sampling is visible in the result: between two stamps the tool is never evaluated, so for $s > 1$ the carved floor is visibly stepped. Because the stamper is a *union of stamps*, refining the step can only add positions to that union, never remove one: its removed volume grows **monotonically** as s shrinks, approaching the continuous cut from below in removed volume. §4.6 returns to how the single-pass swept subtraction relates to that limit and to the stamper.

Stamping (baseline) vs Swept subtraction

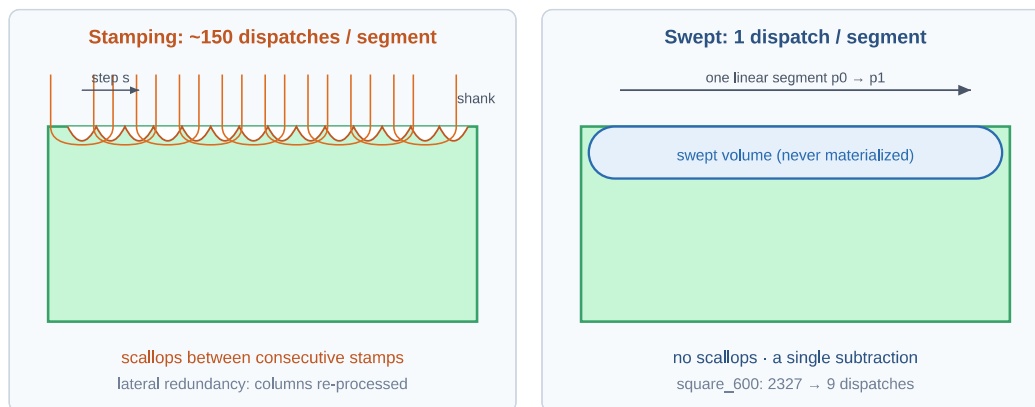


Figure 2. Stamping versus swept subtraction. Left: many overlapping stamps along the path — a coarse step leaves a stepped floor and re-processes each column many times (the tool is a ball-end mill: hemisphere tip plus shank). Right: a single subtraction of the swept volume, one pass with a step-free floor. On the square_600 benchmark the dispatches drop from 2327 to 9.

3.2 Swept-segment subtraction

Instead of L/s stamps we perform **one subtraction per linear segment**, of the tool's **swept volume** (the Minkowski sum $\text{tool} \oplus p_0p_1$). Set difference is order-

independent and distributes over unions, so subtracting the swept volume once equals stamping at every sub-position of the Chebyshev grid along the segment (§4.6). Effect: on square_600 the dispatches drop from 2327 to 9.

3.3 On-the-fly Z-envelope, fused in one kernel (core contribution)

The subtlety is that “subtract the swept volume” would seem to require **building** it in a separate buffer and then subtracting it. We never do. Because the tool’s XY columns are single solid intervals, moving it by ≤ 1 voxel per sub-step makes the intervals a given stock column “sees” **overlap**; the union of overlapping intervals is still one interval [min bottom, max top] - the **Z-envelope**. A single fused compute shader computes this envelope on the fly, per column, and subtracts it in place in the same pass. We quantify what this fusion buys against the “seemingly required” external-buffer version - a full second buffer plus an extra pass - in §6.1 ($\approx 1.9\times$ faster) and §6.3 (half the resident memory).

Per-column Z-envelope, computed on the fly (the core of the method)

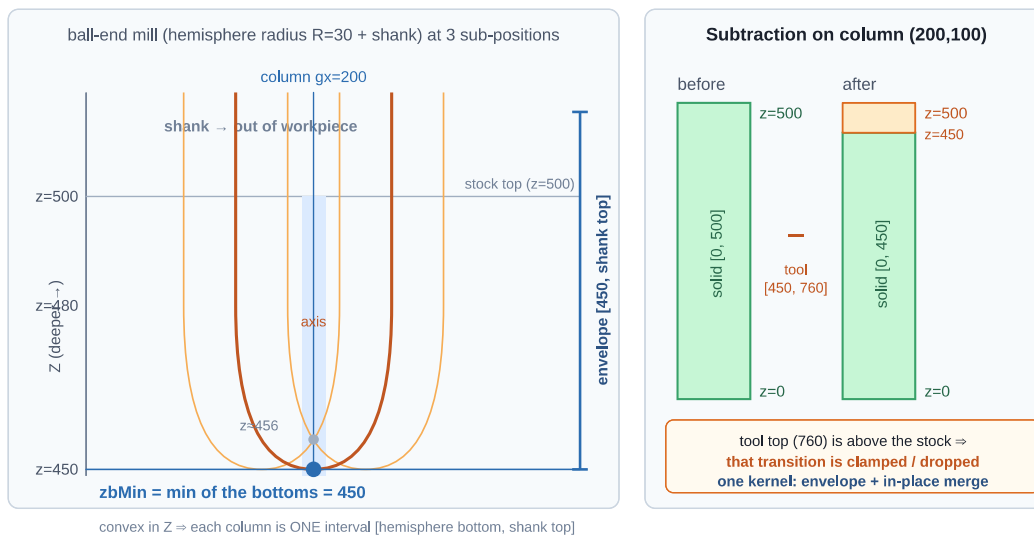


Figure 3. The per-column Z-envelope. Left: the ball-end mill (hemisphere tip + shank) at three sub-positions; the shank rises out of the workpiece. For a given column the deepest bottom is reached when the tool axis passes over it; the envelope is [zbMin, shank top]. Right: its subtraction from the column, done in place in the same kernel - the tool’s top transition, above the stock, is clamped.

Worked example. A stock column starts as the single pair [0, z_top]. As the tool reference point slides in X, the column is covered for a contiguous sub-range of sub-positions; for each, the hemisphere bottom at that lateral offset is computed. The envelope takes the minimum bottom (deepest, on-axis) and the maximum top

(the shank top, above the stock). Subtracting $[z_bMin, shankTop]$ from $[0, z_top]$ - with the shank-top transition clamped to the stock range - yields $[0, z_bMin]$: exactly what stamping at every sub-position of that range would produce, via a short loop and one merge, and with no buffer for the swept volume.

3.4 Sub-step bounding

A column is covered by the tool only for a contiguous range of sub-steps $[k0, k1]$, obtainable in closed form from the tool centre's linear motion. Iterating only that range (with ± 1 safety margins) drops the per-column work from *segment length* to $\approx$ *tool extent* on axis-aligned moves.

3.5 GPU compaction for read-back

After carving, the result must be returned in the compact column format. Reading back the unpacked working buffer would be $W \cdot H \cdot 32 \cdot 4 \text{ B} \approx 128 \text{ MB}$. Instead we compact **on the GPU** first (§4.2), so only $\sim 8 \text{ MB}$ of dense transitions cross the bus; read-back drops from $\sim 72 \text{ ms}$ to $\sim 8 \text{ ms}$.

3.6 Tube pruning (the new optimization)

The dispatch (§4.3) covers the **axis-aligned bounding box (AABB)** of the two **tool-centre endpoints, dilated by the tool footprint**. For an axis-aligned move that AABB already equals the swept band, but for a **diagonal** move of length L and tool footprint F it is $\approx (0.7L + F)^2$ columns while the tool sweeps only $\approx L \cdot F$ - and every over-covered column still rewrote all 32 slots for a no-op, because the shader had no early-out before its write-back. In-air rapid (G0) moves are the same: the tool is above the stock, so every column is a no-op that still rewrites 32 slots.

Tube pruning: dispatch AABB vs. the swept band

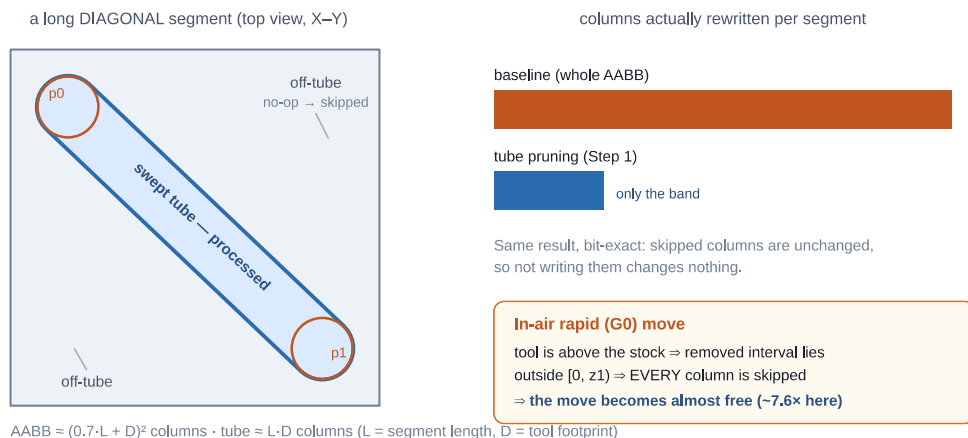


Figure 4. Tube pruning. On a long diagonal segment the dispatch AABB (dashed) is far larger than the swept band it contains; the off-band columns are no-ops that were still being rewritten. Skipping their write-back leaves them untouched (byte-identical) and removes that memory traffic. In-air rapids are the extreme case: the removed interval is entirely outside the stock's Z-range, so every column is skipped.

The fix is a single early-out in the shader (§4.4): before the merge and write-back, return when the tool never covers the column (`hasMat == false`) or the removed interval lies entirely outside the stock's Z-range (`zbMin ≥ z1 || ztMax ≤ 0`). The column is left untouched - correct, because it is unchanged - and its 128 bytes of write traffic vanish. This is what makes over-dispatched diagonals and in-air rapids cheap; axis-aligned dispatches are fully covered and unaffected.

Segment-level pruning. The same idea applies one level up, on the host: when the tool's Z-extent over a *whole* segment cannot reach the stock's `[0, z1)` range - the common case for an in-air G0 rapid (which maps below the stock under the inverted-Z convention) - the segment removes nothing and its **entire dispatch is skipped**, not merely pruned per column. Rapids become essentially free (a synthetic all-rapid program: ~2900×) and every program with repositioning moves speeds up; it is host-side, byte-identical, and gated by the same toggle.

4. Algorithm and GPU implementation

4.1 A short GPU-compute primer

The pipeline uses **compute shaders** - programs that run on the GPU outside the graphics pipeline. A program is launched by a **dispatch** that creates a grid of **workgroups**; each workgroup contains a fixed block of **threads** (here `local_size = 8 × 8 × 1`, so 64 threads) that execute the shader in parallel. Each thread reads its `gl_GlobalInvocationID`, a 3-D index identifying it within the whole grid. Data lives in **Shader Storage Buffer Objects (SSBOs)** - large GPU-resident arrays the shader reads and writes. Ordering between dependent dispatches is enforced by a **memory barrier**. In our carver **one thread processes one workpiece column**; this is what makes the per-column-merge property of the dixel representation (§2) map directly onto the hardware.

4.2 Data layout: the flat working buffer

During a carving session the stock is held GPU-resident in a fixed-stride **flat** buffer, `obj1_flat`, of `W·H·MAX_TRANSITIONS` uints (`MAX_TRANSITIONS = 32`), plus a per-column count buffer `obj1_dataNum`. Column `(x,y)` owns the 32-slot slice

obj1_flat[idx*32 ... idx*32+32), idx = x + y·W. Subtractions write results **in place** into the same slots, so no per-operation reallocation is needed. This buffer is 1000·1000·32·4 B = 128 MB for the reference stock - the working set whose bandwidth dominates the cost (\$7). The tool is uploaded once as a compact (compressedData, prefixSumData) pair and never mutates.

4.3 Dispatch: mapping threads onto the XY grid

For each segment the host computes the swept bounding box - the AABB of the two tool-centre endpoints, dilated by the tool half-footprint ($w2/2$, $h2/2$) and clamped to the grid - giving an origin (baseX, baseY) and a size (bboxW, bboxH). It then dispatches $\lceil \text{bboxW}/8 \rceil \times \lceil \text{bboxH}/8 \rceil$ workgroups. Inside the shader each thread recovers its column as $gx = \text{baseX} + gl_GlobalInvocationID.x$, $gy = \text{baseY} + gl_GlobalInvocationID.y$, and returns immediately if $gx \geq W$ or $gy \geq H$ (the ragged edge of the last workgroups).

Mapping compute threads onto the workpiece column grid

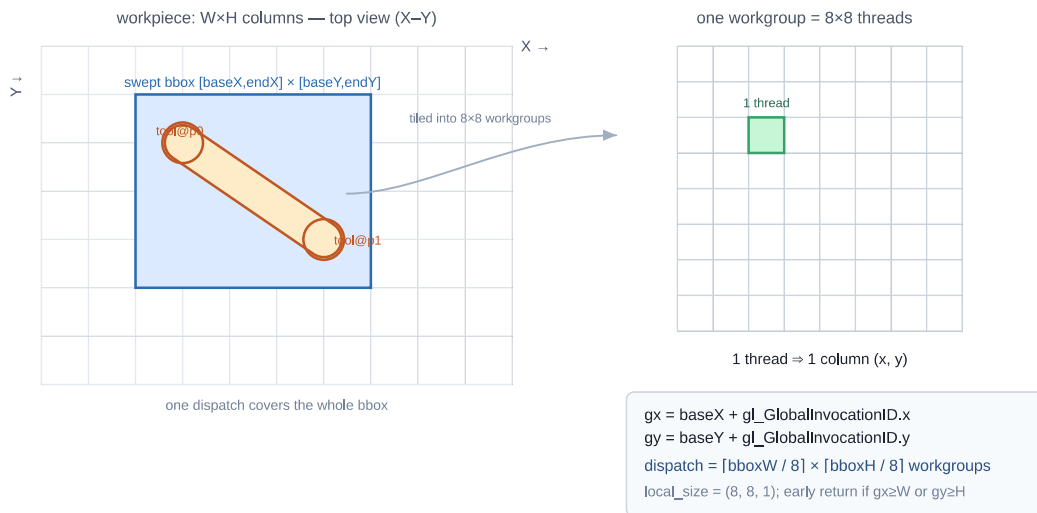


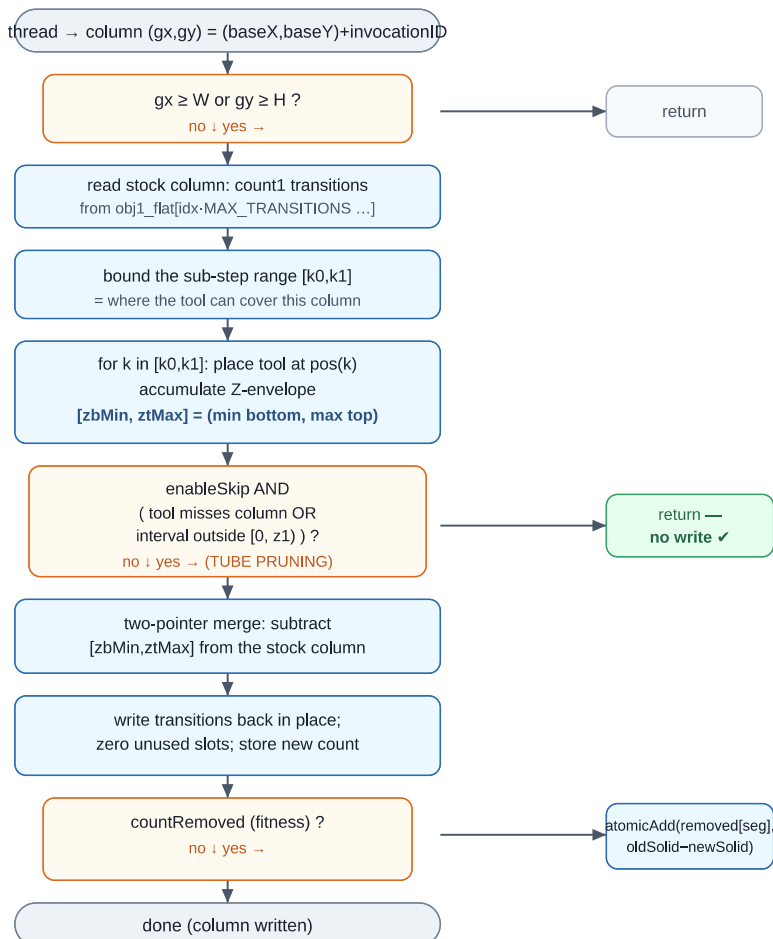
Figure 5. Mapping compute threads onto the workpiece column grid. The host restricts the dispatch to the swept bounding box; the GPU tiles it into 8×8 workgroups, and each thread handles one column (x,y) recovered from its global invocation ID. The number of workgroups is $\lceil \text{bboxW}/8 \rceil \times \lceil \text{bboxH}/8 \rceil$.

4.4 The subtract_swept shader, step by step

Each thread runs the following (Figure 6). It is deliberately branch-light and reads the tool from cached buffers while touching the large obj1_flat buffer exactly once per column.

1. **Identify the column** and bounds-check it against W, H .
1. **Read the stock column** - its transition count $count1$ from $obj1_dataNum$, its transitions from $obj1_flat$.
2. **Bound the sub-step range** $[k0, k1]$ - the sub-positions of the tool centre for which the tool could cover this column (from the linear motion $tr \approx start + k \cdot \Delta/K$).
3. **Accumulate the Z-envelope** - loop k over $[k0, k1]$; for each, find the tool's column under (gx, gy) and update $zbMin = \min(zbMin, bottom)$, $ztMax = \max(ztMax, top)$.
4. **Tube-pruning early-out** - if $enableSkip$ and (the tool never covered the column, or the interval $[zbMin, ztMax]$ lies entirely outside $[0, z1]$), return **without writing** (§3.6).
5. **Subtract** - a two-pointer merge of the stock column with the single interval $[zbMin, ztMax]$, emitting a transition whenever the *result* occupancy (stockInside AND NOT toolInside) changes.
6. **Write back** the result transitions in place, zero the unused slots, and store the new count.
7. **Optional counter** - when material-removal tracking is on (used by the fitness evaluator), $atomicAdd(removed[segment], oldSolid - newSolid)$.

subtract_swept.comp — per-thread flow (one column)



Blue = compute step · Orange = branch · Green “no write” = the tube-pruning early-out (Step 1).
Every column of the swept bbox runs this independently, one GPU thread each.

Figure 6. Per-thread control flow of `subtract_swept.comp`. Blue boxes are compute steps, orange are branches; the green “no write” return is the tube-pruning early-out. Every column of the swept bounding box runs this independently, one GPU thread each.

Two design points matter for performance. The **merge is in place** and touches the 32-slot column once; and the **envelope reuses the column’s lowest/highest transition** rather than an intermediate swept solid. The tube-pruning early-out (step 5) is the only change of §3.6, and it removes the write of step 7 - the expensive part - for no-op columns.

4.5 Tool geometry

A ball-end mill is not a sphere but a **cylinder with a hemispherical tip**; its shank rises above the stock and, in normal 3-axis milling, is out of the workpiece at every instant. This is compatible with the single-interval property: the hemisphere-plus-shank solid is convex in Z, so each of its XY columns is one interval [hemisphere bottom, shank top]. The shank top is a genuine transition in the tool's representation, but because it lies above the stock it is clamped away by the $[0, z_1)$ bound during subtraction. The method is exact for tools whose columns are single Z-intervals (ball, flat, conical, bull-nose) and over-removes gracefully for non-convex tools — an artefact of collapsing the tool column to a single [bottom, top] interval (§4.4); the full-transition-list stamper does not share it.

4.6 Correctness: what is byte-identical, and what is exact

Two distinct guarantees hold, and it is worth separating them. **(i) The swept formulation is exact with respect to stamping on the same sample grid.** Set difference distributes over unions and is order-independent, so $A \setminus (B_1 \cup \dots \cup B_n) = (\dots((A \setminus B_1) \setminus B_2) \dots \setminus B_n)$; the swept volume is the union of the tool at the sub-positions sampled along the segment, and sampling those at Chebyshev step $K = \max(|\Delta x|, |\Delta y|, |\Delta z|)$ (the L_∞ length) guarantees ≤ 1 -voxel motion per sub-step, so the envelope touches every integer position a stamper *on that grid* would. Subtracting the envelope once therefore equals stamping at those positions. **(ii) The swept optimizations are byte-identical to one another.** The external-buffer (S1), fused (S2) and tube-pruned (S3) kernels produce byte-identical carved output — pruning only skips writes to columns it leaves unchanged — which is the load-bearing correctness claim: it shows the fusion and the pruning change nothing geometric. We verify it directly by comparing the saved .bin files (§5.3).

The --legacy stamper is a coarser reference and its removed volume differs from the swept result by a small amount whose *sign and mechanism* are worth stating precisely, since a stray comparison misleads. Two effects are in play. **First, an implementation artifact:** the reference's drive loop advanced before it stamped, so it skipped each segment's *starting* sample — an off-by-one that under-removed the entry cap. We corrected it (the start is stamped once, up front); on a single axis-aligned segment this closed $\approx 85\%$ of the discrepancy, and it does not affect the timing figures (one extra stamp per program — the first toolpath point).

Second, and genuinely, the swept subtraction slightly *under-removes* relative to the continuously-swept cut. A stamper is a union of stamps, so a finer step only adds positions and removes monotonically more, converging to the continuum; the swept samples the tool centre at integer positions, so a column whose deepest

coverage falls at a *fractional* centre is cut a fraction of a voxel shallower than the true minimum. Measured on one clean segment against a quarter-voxel stamper (a continuum proxy), this under-removal is on the order of **0.05 %** of the segment's removed material on an axis-aligned move and **0.15 %** on a 45° move, growing as the reference step shrinks — the expected behaviour of a single-pass approximation. So of the two discretizations the fine-sampled limit is the more faithful, and the swept is a measured under-approximation of it, below 0.2 %: a modest, quantified error, not an ambiguity. We therefore use S0 as a timing and behavioural reference, not a geometric ground truth.

The correctness the paper *does* rest on is the byte-identity of (ii), checked by a direct diff of the saved .bin files, together with the **residual solid volume** invariant — the sum over all columns and their solid intervals of (top – bottom), a deterministic, render-independent quantity — used for the volume comparisons above.

5. Numerical experiments

5.1 Hardware, software, and the A/B toggle

Measurements are on an **Intel Lunar Lake integrated GPU** (Mesa iris driver), OpenGL 4.6 compute, which autocam reaches via an **EGL/Wayland** context. (Note: the X11 glxinfo utility reports llvmpipe on this system, but that is the GLX path, not the context autocam uses; the real driver is confirmed from the EGL log.) Every refinement level runs from the **same binary**, selected at runtime - essential for a fair A/B: --legacy is per-step stamping (S0); --legacy-external-buffer is the two-pass materialize-then-subtract swept (S1); the fused kernel with AUTOCAM_SWEPT_SKIP=0 is the fused in-place swept (S2), and =1 (default) adds tube pruning (S3). All runs are headless (--no-view), and timings are the interleaved min-of-N described in §5.4.

5.2 Benchmarks and the G-code generator

Because the optimization targets segments whose length L exceeds the tool footprint F , the benchmark set must exercise that regime - long, non-axis-aligned segments and air travel - which the project's original programs (a square perimeter, star pocket, rectangular pocket) do **not** when run with the wide 256-voxel hemispheric_mill_10. We therefore use:

- square_600 with hemispheric_mill_10 (256 wide) - an **axis-aligned control** (AABB already equals the tube; expected $\sim 1\times$).

- bench_complex with hemispheric_mill_3 (32 wide) - a **45° raster** with 13 long diagonal cutting passes interleaved with long diagonal air repositions, generated programmatically.
- Two stress micro-benchmarks with the 32-wide tool: diag_cut (repeated long diagonal cuts) and air_moves (repeated long diagonal G0 rapids).

The 45° raster is emitted by a small parametric generator that walks lanes in a rotated basis (cutting direction $\hat{u} = (1,1)/\sqrt{2}$, step direction $\hat{v} = (1,-1)/\sqrt{2}$), converting each (u,v) to (x,y); each lane is a long diagonal G1 cut followed by a retract and a long diagonal G0 reposition to the next lane. This mirrors a real finishing raster while maximizing both effects the optimization targets.

5.3 Validation

For every configuration we save the carved .bin. Across the three **swept** levels — external-buffer S1, fused S2, pruned S3 — the file is **byte-identical**, which is what certifies that the fusion and the pruning leave the geometry untouched; the comparison is a direct diff of the files. The stamper S0 is a coarser, differently-sampled reference (§4.6), not a ground truth: it is not byte-identical to the swept output at any step we tested (2, 1, 0.5, 0.25 voxels), and against a fine (quarter-voxel) stamper — a proxy for the continuum — the swept *under*-removes by the order of 0.05 % of a segment’s removed material on an axis-aligned move and 0.15 % on a 45° move, growing as the step shrinks (§4.6). We also confirm the residual solid volume against the project’s reference numbers (square_600 = 281 165 072, star_pocket = 462 722 118), computed offline by an independent pure-Python reader (no dependency on the GPU path).

5.4 Methodology

Each configuration is run several times; the **first run is discarded** as compute-pipeline warm-up and the **minimum** net-carving time is reported. Crucially, the modes are **interleaved per round** (one run of each mode per round) rather than measured in separate batches, so that GPU clock/thermal drift cancels in the ratio - without this, the never-taken branch of the axis-aligned control spuriously appears 30% slower. Timing uses the simulator’s internal “net carving” timer (a glFinish around the per-segment loop, excluding read-back).

6. Results

We report two phases of optimization, and for each we give both **speed** and **memory**. **Phase 1** turns the naive per-step stamper into the fused swept baseline - $\approx 31\times$ faster *and* a smaller working set (Table 2, §6.3). Within Phase 1 the cross-

workload matrix (§6.1, Table 3) separates the part the literature already delivers (the per-move swept formulation, S0→S1) from **this work’s fusion** of the subtraction (S1→S2) - measured on its own by A/B-ing against an external-buffer implementation. **Phase 2** is this work’s tube-pruning + in-air skip on top of that baseline (S2→S3, Tables 1 and 3), plus the sparse tiled working set (§6.2). The primary comparison is the matrix of §6.1; Table 1 first isolates the Phase-2 pruning lever on adversarial stress benchmarks - an axis-aligned control plus hand-built diagonal and air-heavy programs. The memory side of both phases is consolidated in §6.3.

Table 1 - Phase 2 pruning on stress benchmarks: net carving time (ms), Intel LNL / iris, interleaved min-of-N, all byte-identical. “S2 swept” is the fused Phase-1 baseline; “S3 +pruning” is this work; “oriented-tile” is the reverted oriented dispatch (a negative result, §7), shown for completeness. (Level labels S0-S3 are defined once in §6.1.)

benchmark (tool)	S2 swept (fused baseline)	S3 +pruning (ours)	oriented-tile (reverted)	S2→S3
square_600 (mill_10, control)	23.7	24.5	28.0	0.97×
bench_comple x (mill_3)	42.7	14.8	23.6	2.89×
diag_cut (mill_3)	31.7	6.2	12.4	5.11×
air_moves (mill_3)	29.6	3.9	7.5	7.59×

Net carving time, normalized to baseline (lower = faster)

Intel Lunar Lake / iris, interleaved min-of-N, bit-exact. Small tool (32) on long non-axis-aligned segments.

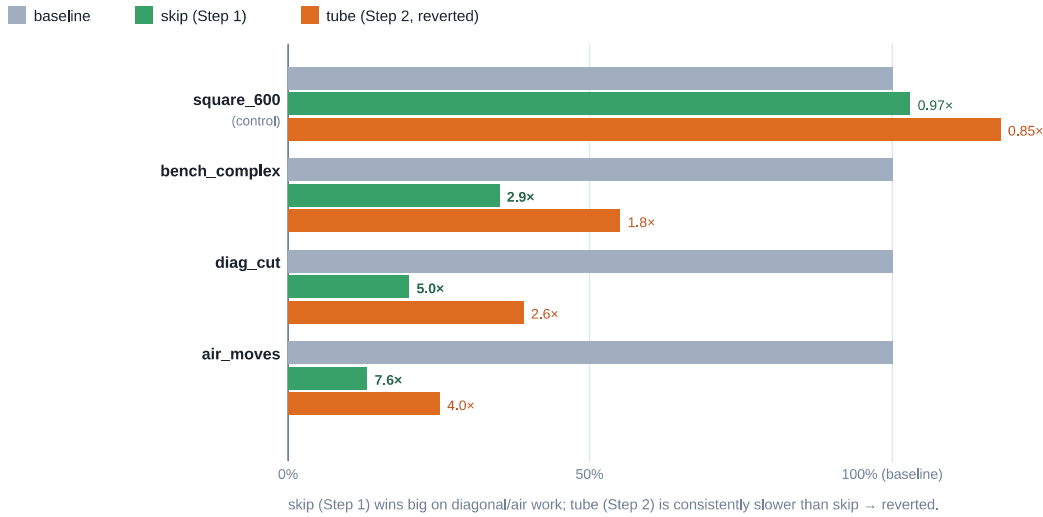


Figure 7. Net carving time normalized to the fused S2 swept baseline (lower is faster). S3 tube pruning wins large on diagonal and air-heavy work and is neutral on the axis-aligned control; the oriented tiled dispatch is consistently slower and was reverted.

For historical context, **Table 2** recalls how the Phase-1 swept baseline itself was reached from the naive stamper — the pre-pruning optimization history on the same class of GPU, on square_600 with the larger mill_10 tool.

Table 2 — Phase-1 optimization history (square_600, mill_10; carving-loop wall time, read-back included). The swept stages (swept → compaction → sub-step) are byte-identical to one another; the stamping baseline is a coarse reference, not a geometric ground truth (§4.6).

Stage	Carving-loop time
Baseline: per-step stamping	~1232 ms
– per-step CPU / sync overhead	~700 ms
Swept-segment subtraction (§3.2)	~114 ms
GPU read-back compaction (§3.5)	~67 ms
Sub-step bounding (§3.4)	~40 ms (net carving ~26 ms)

Overall ~1232 → ~40 ms (≈31×). The Phase-2 pruning of §3.6 is applied on top of that swept baseline. Because Table 2 uses square_600 with the larger mill_10 tool

and counts read-back, its absolute milliseconds differ from the mill_3 net-carving figures in Tables 1 and 3; the comparison of interest is always within a table, on one binary and hardware.

6.1 Cross-workload matrix: established methods vs. this work

Table 1 tracks one program; to separate what a *known* method already buys from this work's contributions, we ran a suite of machining workloads that stress different axes at **four** refinement levels, all on the **same binary, representation and GPU** - apples-to-apples. We deliberately do **not** quote other authors' absolute timings (meaningless across hardware, representation, tool and resolution); each level is *our* fair, optimized implementation of the corresponding algorithm class, anchored to the literature it represents:

- **S0 - per-step stamping:** the classic z-map/dexel display method (Van Hook 1986 [1]); the tool is stamped at fixed jog steps along each segment (--legacy, default step 2 voxels — the setting a display-oriented stamper would realistically use). S0 is a timing and behavioural reference, not a geometric ground truth (§4.6).
- **S1 - swept, external buffer:** the straightforward per-move swept implementation (the multi-/tri-dexel class; Müller & Surmann 2003 [2]; Tukora & Szalay 2012 [3]; Inui et al. 2019 [4]) - the swept volume is **materialized in a separate full-size buffer** and then subtracted (--legacy-external-buffer).
- **S2 - swept, fused in-place: this work's fusion** - the swept-tool column is computed on the fly as a Z-envelope and subtracted **in place in the resident buffer**, with **no intermediate swept-volume buffer** (§3.3; AUTOCAM_SWEPT_SKIP=0).
- **S3 - + tube pruning + in-air skip:** this work's Phase-2 traffic pruning on top of S2 (§3.6).

The design decision the paper claims - fuse the subtraction rather than materialize the swept volume - is thus isolated as a single measured step, **S1→S2**. The three swept levels (S1–S3) are byte-identical to one another; S0 is a timing and behavioural reference, not a geometric ground truth (§4.6).

Table 3 - carving netto (ms) per machining workload and refinement level (small hemispheric_mill_3 tool; min of 5 interleaved runs, Intel iris; swept levels byte-identical, S0 a coarse reference not a ground truth). **S1→S2** is the fusion / no-external-buffer gain (this work); **S2→S3** is pruning (this work); **S0→S3** is overall:

Workload (what it stresses)	S0	S1	S2	S3	S1→S2	S2→S3	S0→S3
<i>what the level is</i>	<i>stamping</i>	<i>swept, ext. buffer</i>	<i>swept, fused</i>	<i>+ prune + air-skip</i>	<i>(fusion)</i>	<i>(prune)</i>	
<i>attribution</i>	<i>Van Hook [1]</i>	<i>straight forward swept [2–4]</i>	<i>this work</i>	<i>this work</i>			
contour — axis-aligned perimeter	47.9	9.26	3.69	4.11	2.51×	0.90×	11.7×
pocket_axis — axis-aligned raster	120.5	16.0	8.23	8.88	1.94×	0.93×	13.6×
raster45 — 45° diagonal raster	182.7	44.4	36.4	11.9	1.22×	3.06×	15.4×
rapids — scattered cuts, long air moves	72.2	16.5	11.1	5.98	1.49×	1.86×	12.1×
localized — small corner feature	33.1	11.9	4.72	4.29	2.52×	1.10×	7.7×

Workload
(what it stresses)

	S0	S1	S2	S3	S1→S2	S2→S3	S0→S3
finishing — fine-stepover raster	93.7	13.8	6.93	6.93	1.99×	1.00×	13.5×

Two of the levels are established, two are ours. **S0→S1 (≈ 2.8 - $7.5\times$)** is the per-move swept formulation over per-step stamping - what the known methods already deliver. **S1→S2 (≈ 1.2 - $2.5\times$, **geomean $\approx 1.9\times$**)** is **this work's fusion**: computing the swept envelope on the fly and subtracting it in place, rather than writing it to a separate buffer and subtracting in a second pass, is a consistent speed win *and* removes a full-size resident buffer (§6.3) - the gain is largest where the carve is otherwise cheap (the fixed cost of the extra pass and buffer round-trip dominates) and smallest on the big diagonal raster. **S2→S3** is this work's tube pruning, workload-dependent by construction: it removes the over-dispatch of *non-axis-aligned* segments (45° raster **3.06×**) and *in-air* rapids (**1.86×**), and on axis-aligned programs it costs 7–11 % (contour 0.90×, pocket_axis 0.93×) — the price of the per-column branch where the AABB already equals the swept band. Fusion (S1→S2) and pruning (S2→S3) are complementary - the first always pays, the second pays where the dispatch over-covers. Reproducible via `tools/bench_matrix.sh`.

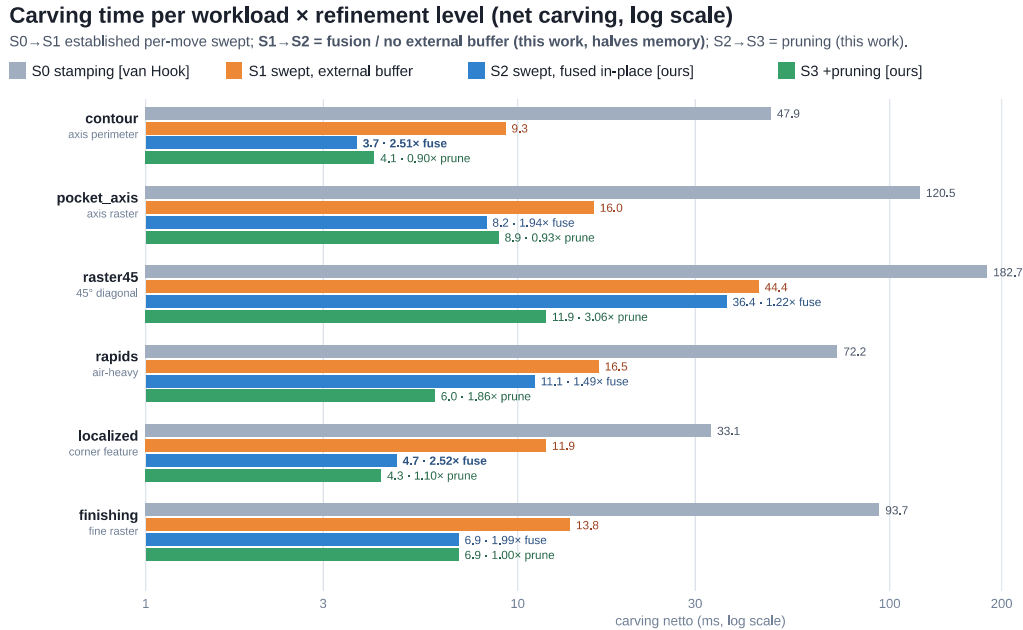


Figure 8. Net carving time per machining workload and refinement level (log scale). S0→S1 is the established per-move swept method; **S1→S2 (orange→blue) is this work’s fusion - the swept volume computed on the fly and subtracted in place instead of via a separate buffer (~1.2-2.5×, and it halves resident memory)**; S2→S3 (blue→green) is this work’s tube-pruning + in-air skip, which pays where the dispatch over-covers (45° raster 3.06×, air-heavy 1.86×).

6.2 Working-set: a sparse tiled study

The 128 MB flat buffer is ~16× padding (MAX_TRANSITIONS = 32 vs. 2-4 real transitions per column). A full alternative backend (AUTOCAM_CARVE_BACKEND=sparse) stores the stock in 32×32-column tiles, keeping untouched tiles UNIFORM (O(1)) and materializing a tile only when a segment’s bounding box first touches it - byte-identical to the flat backend, runtime-selectable. It isolates two effects. **(i)** Per-carve bandwidth is bounded by the *swept bounding box*, not the buffer size (both backends touch only the bbox columns), so tiling does not reduce the carve’s traffic *volume*; it only improves access *locality* (the strided-access problem) → ~1.1× (square_600 1.09×, bench_complex 1.11×, pocket 1.15×). **(ii)** Footprint is a conditional win tied to locality - a localized program materializes 49-56 / 1024 tiles (**8 MB** vs. 122 MB, ~15×), whereas whole-stock machining touches most tiles (raster45 736 / 1024, no saving). So sparse tiling is a **memory/scale** lever (finer resolutions, larger stocks, smaller read-back for localized work), not a carve-speed lever; host-side materialization even makes whole-stock programs slower.

6.3 Memory and data movement across the two phases

Speed is only half the story: each phase also shrinks what the GPU must **hold** and **move**. Table 4 consolidates the footprint and data-movement figures introduced piecemeal above (§3.3, §3.5, §6.2) for the reference 1000×1000×500 stock.

Table 4 - Resident footprint and data movement, per phase (reference stock).

	Naive stamping	Swept, external buffer (S1)	Phase 1 fused (S2)	Phase 2 (this work, S3)
Material-to-remove	tool stamped every jog step	materialized - swept volume written to a buffer	none - O(1) Z-envelope per column, on the fly	same as S2
Resident working set	128 MB flat	256 MB - 128 MB stock + a 128 MB swept twin	128 MB (no twin)	8 MB sparse for localized work (~15×), else 128 MB
Result assembly	separate output + per-step 12 MB stock copy	second pass reads the twin, subtracts	in place in the one working buffer	same as S2
Read-back transfer	128 MB unpacked	8 MB GPU-compacted	8 MB GPU-compacted (~16×)	same as S2
Per-carve write traffic	whole grid, full footprint	swept-AABB columns, twice (write twin + subtract)	swept-AABB columns, once	swept tube only - off-band / off-Z columns skipped

The fusion (S1→S2) is a **memory** decision as much as a speed one: the external-buffer baseline needs a full-size 128 MB **twin** of the working buffer to hold the

materialized swept volume ($2\times$ resident), and round-trips the envelope through it, whereas the fused kernel keeps only an $O(1)$ per-column envelope in registers - so it **halves resident memory** while also running $\approx 1.9\times$ faster (§6.1). Phase 1's memory wins are otherwise **unconditional** (in-place single buffer, no per-step 12 MB stock copy, 8 MB compacted read-back); Phase 2 adds a **conditional** footprint win tied to spatial locality (§6.2) and a **write-traffic** reduction tied to non-axis-aligned / in-air motion (§6.1). Neither phase trades memory for speed: the two move together, because the simulator is bandwidth-bound (§7).

7. Discussion

The bottleneck is memory-write bandwidth. A design A/B and three follow-up experiments, read together, localize it:

- *Fusion vs. an external buffer (positive, §6.1).* Materializing the swept volume in a separate full-size buffer and subtracting it in a second pass round-trips the envelope through memory (write the twin, read it back) and doubles the resident set. Fusing the two - envelope on the fly, subtracted in place - removes that traffic and the twin, for $\approx 1.9\times$ and half the memory. Removing *memory traffic* pays; the swept-envelope *compute* is identical in both, and it is not where the time goes.
- *Range-query envelopes (negative).* Precomputing per-tool directional Z-envelopes as range-minimum/maximum sparse tables makes the per-column envelope an $O(1)$ query instead of an $O(\text{sub-steps})$ loop. Implemented exactly, it gave only $\sim 25\text{-}30\%$ - because it optimized *compute*, which is not the bottleneck - and was reverted.
- *Tube pruning (positive, §3.6).* Skipping the write-back of off-band and out-of-Z-range columns attacks *memory-write traffic* and gives $2.9\text{--}7.6\times$ in its regime (stress benchmarks, Table 1). This is the complement of the previous result: the lever that pays removes memory traffic, not compute.
- *Oriented tiled dispatch (negative).* Tempted to also avoid *launching* the off-band threads (each still does a counter read plus the early-out), we tiled the dispatch along the motion so it follows the band. Each tile reuses the full-segment envelope uniforms (so every column is computed identically); adjacent tiles overlap idempotently, separated by a barrier - keeping it byte-identical. The result was $\sim 2\times$ **slower** than tube pruning alone: the per-tile barriers serialize the tiles and the joins double-process columns, and that synchronization cost exceeds the thread-launch saving, because tube pruning already made the off-band threads cheap (a read, no write). Reverted.

- *Sparse tiled working-set (footprint, not speed; §6.2).* A tiled backend that keeps untouched tiles UNIFORM shrinks the working set to the touched fraction, but the per-carve traffic is bounded by the swept bounding box in either layout, so it only improves access *locality* ($\sim 1.1\times$) and reduces *footprint* conditionally on locality ($\sim 15\times$ localized, nothing whole-stock). Working-set size is a memory/scale lever, not a bandwidth one.

The pattern is consistent - a **map of the bottleneck**. Attacking *compute* (RMQ), *launched-thread count* (tiled dispatch) or *working-set size* (sparse tiling) yields diminishing, negative or footprint-only returns; the two levers that pay both remove *memory traffic on the over-dispatched bounding box* - **fusing away the swept-volume round-trip** (S1→S2, always) and **tube pruning + in-air skip** the off-band/off-Z write-back (S2→S3, where the dispatch over-covers). The simulator is **bbox-bandwidth-bound**: the productive optimization is the one that removes bbox traffic. A dispatch-enqueue cost that stays ~ 0.4 ms whether a program has 9 or 202 segments corroborates that per-segment host overhead is not the limit.

Portability and the roofline argument. All measurements here are on one integrated GPU, which invites the question of whether the findings are hardware-specific. They are not, and the reason is structural rather than empirical. The per-column merge moves on the order of 128-256 bytes per active column - it rewrites all MAX_TRANSITIONS = 32 slots and reads a comparable number - while performing only a few integer comparisons per 4-byte transition: an **arithmetic intensity well below 1 op/byte**, one to three orders of magnitude under the ridge point of any modern GPU roofline. A kernel this far into the memory-bound regime is memory-bound on *every* GPU; and because discrete NVIDIA parts carry a *higher* compute-to-bandwidth ratio than an integrated Intel GPU, the diagnosis becomes **more** true there, not less. The load-bearing claims are therefore architecture-invariant: the representation, the exactness argument, the “carving is a memory-movement problem” thesis, and the *geometric* workload dependence of tube pruning (a function of the AABB-to-swept-band ratio, not the silicon) all transfer unchanged. The pipeline also uses only portable compute primitives (SSBOs, std430, dispatch, a storage barrier) - no subgroup intrinsics, tensor or ray-tracing cores - so a Vulkan or CUDA port is a direct transliteration of the same kernel.

A faster or discrete GPU changes *magnitudes*, not directions. Absolute carve times fall with bandwidth while the kernel stays bandwidth-bound; the fusion and pruning ratios shift because their **fixed** components (an extra pass and a barrier for fusion; a per-column branch for pruning) scale differently from the traffic

they save - the fusion win is already largest where the carve is cheapest, and that fixed-cost fraction only grows as traffic gets faster. One optimization would become **more** valuable off the iGPU: the GPU-side read-back compaction (128 MB → 8 MB), because on a discrete card the read-back crosses PCIe (~16-32 GB/s), where the full unpacked buffer alone would cost several milliseconds.

Two caveats are genuinely API- or occupancy-specific and would warrant re-measurement - none of them touching the central result. First, the *oriented tiled dispatch* negative result is partly a statement about OpenGL's coarse glMemoryBarrier and dispatch model: CUDA and Vulkan expose finer synchronization (timeline semaphores, cooperative groups, persistent kernels, dynamic parallelism) that could avoid the barrier serialization that sank it, so that reverted idea may be worth revisiting on those APIs. Second, the per-thread localOut[64] scratch (256 B) and the branchy sub-step loop interact with the register file and 32-wide warps: register spilling or warp divergence could move the fine ratios, and the large L2 caches of recent discrete GPUs (tens of MB) might hold more of the active bounding box, plausibly *increasing* the sparse-tiling locality benefit. These set the agenda for a cross-hardware study; they do not change which lever breaks the ceiling.

When tube pruning helps. The gain requires segment length $L \gg F$. Axis-aligned programs, or diagonal programs machined with a tool nearly as wide as the segments are long, see little - their AABB already equals the tube. Real 3-axis programs, however, contain substantial air travel (rapids between features, retract/reposition links) and non-axis-aligned finishing passes, all of which this optimization accelerates; the in-air pruning in particular applies to **every** rapid.

Limitations. The pipeline runs in voxel units and the tool datum is the centre of the tool's bounding box, so the Z origin shifts with the modelled shank length and neither the tip radius nor the tool length is compensated; a calibrated millimetre mapping with an explicit tool datum is future work. The envelope over-removes for non-convex-in-Z tools, and the flat-buffer stride MAX_TRANSITIONS = 32 is not overflow-guarded. The tool is axis-fixed (3-axis only). Against the continuously-swept cut the single-pass envelope under-removes by a measured 0.05–0.15 % of a segment's removed material, growing on non-axis-aligned moves (§4.6). The largest remaining lever for the optimization loop is a **GPU-side objective** (comparing carved-vs-target on the GPU) to remove the per-iteration read-back entirely.

8. Conclusions

We presented a GPU material-removal simulator for 3-axis milling built on a single-axis dixel field, in which Boolean difference is a per-column merge that maps one workpiece column to one GPU thread. On top of a fused, materialization-free swept-segment subtraction, we introduced **tube pruning** - skipping the write-back for columns the swept tool never touches - which is byte-identical and yields 2.9–7.6× (stress benchmarks, Table 1) on the diagonal and air-heavy work that dominates real toolpaths, while remaining near-neutral on axis-aligned programs. The optimization, and two documented negative results bracketing it, together show that on this pipeline the decisive resource is **memory-write bandwidth**: the successful lever removes bytes written, not arithmetic or launched threads. The simulator is fast enough, and accurate to within a measured under-removal below 0.2 % of the continuously-swept cut (§4.6), to serve as the forward model of a search- or learning-based toolpath generator, which is the subject of ongoing work.

Appendix A. Terminology

Term	Meaning in this paper
Voxel	A cell of the regular 3-D grid; solid or empty.
Dixel	“Depth element”: the run of solid voxels along one axis in a column.
Single-axis dixel field	The stock/tool stored as per-(x,y)-column Z-runs (our representation).
Column	One (x, y) stack of D voxels along Z.
Lattice ($W \times H \times D$)	The voxel grid: $W \times H$ columns, each D voxels deep (D = lattice depth, distinct from the footprint F).
Transition	A Z height where a column’s occupancy flips (empty ↔ solid).
Parity walk	Recovering occupancy by counting transitions $\leq z$: odd $\Rightarrow$ solid.
Stock / workpiece	The block being machined (obj1).
Tool	The cutter (obj2); here a ball-end mill (hemisphere + shank).
Footprint (F)	The tool’s XY extent in voxels ($w_2 \times$

Term	Meaning in this paper
	h2).
Segment	One straight toolpath move $p_0 \rightarrow p_1$.
Swept volume	The region the tool sweeps over a segment (tool $\oplus$ p_0p_1).
Z-envelope	Per stock column, the single interval [min bottom, max top] of the swept tool.
Sub-step	One sampled tool-centre position along a segment.
Chebyshev length (K)	$\max(\Delta x , \Delta y , \Delta z)$; number of ≤ 1 -voxel sub-steps.
AABB	Axis-aligned bounding box (of a segment's endpoints, dilated by the footprint).
Swept band / tube	The columns the tool actually sweeps - a band along the motion.
Tube pruning	Skipping the write-back for columns outside the tube / stock Z-range.
Thread	One GPU work-item; here it processes one column.
Workgroup	A block of threads executed together (8×8 here).
Dispatch	A launch of a compute shader over a grid of workgroups.
<code>gl_GlobalInvocationID</code>	A thread's global index within the dispatch grid.
SSBO	Shader Storage Buffer Object: a large GPU-resident array.
Memory barrier	A synchronization that orders dependent GPU memory accesses.
<code>MAX_TRANSITIONS</code>	Fixed per-column slot count (32) of the flat working buffer.
Byte-identical	Identical saved .bin files across the swept levels (S1–S3); a direct file diff.

Term	Meaning in this paper
	(We avoid “bit-exact”).
Exact	Equal to stamping on the same Chebyshev sampling grid (§4.6) — a mathematical identity, not a file comparison.
Residual solid volume	Σ (top - bottom) over all columns’ solid intervals.
Net carving time	GPU carving-loop time, excluding read-back (measured with a glFinish).

References

1. T. Van Hook. *Real-time shaded NC milling display*. Computer Graphics (Proc. SIGGRAPH ’86), 20(4):15-20, 1986.
8. H. Müller, T. Surmann, M. Stautner, F. Albersmann, K. Weinert. *Online sculpting and visualization of multi-dexel volumes*. Proc. 8th ACM Symposium on Solid Modeling and Applications (SM ’03), 258-261, 2003.
9. B. Tukora, T. Szalay. *Multi-dexel based material removal simulation and cutting force prediction with the use of general-purpose graphics processing units*. Advances in Engineering Software, 43(1):65-70, 2012.
10. M. Inui, M. Kobayashi, N. Umezu. *Cutter engagement feature extraction using triple-dexel representation workpiece model and GPU parallel processing*. Computer-Aided Design and Applications, 16(1):89-102, 2019.
11. C. C. L. Wang, Y.-S. Leung, Y. Chen. *Solid modeling of polyhedral objects by Layered Depth-Normal Images on the GPU*. Computer-Aided Design, 42(6):535-544, 2010.
12. D. Blackmore, M. C. Leu, L. P. Wang. *The sweep-envelope differential equation algorithm and its application to NC machining verification*. Computer-Aided Design, 29(9):629-637, 1997.
13. K. Abdel-Malek, J. Yang, D. Blackmore, K. Joy. *Swept volumes: foundation, perspectives, and applications*. International Journal of Shape Modeling, 12(1):87-127, 2006.
14. W. E. Lorensen, H. E. Cline. *Marching cubes: a high resolution 3D surface construction algorithm*. Computer Graphics (Proc. SIGGRAPH ’87), 21(4):163-169, 1987.
15. C. Dyken, G. Ziegler, C. Theobalt, H.-P. Seidel. *High-speed marching cubes using HistoPyramids*. Computer Graphics Forum, 27(8):2028-2039, 2008.